Exception Handling — Wrong HTTP Status Codes, Confirmed and New

InstantMudra Enterprises Admin Portal

⚠ Two custom exceptions have no specific handler — both fall through to a generic 400

ResourceNotFoundException (confirmed, top-level finding)

```
@ResponseStatus(HttpStatus.NOT_FOUND)
class ResourceNotFoundException extends RuntimeException
```

No @ExceptionHandler(ResourceNotFoundException.class) exists — GlobalExceptionHandler's broader **@ExceptionHandler(RuntimeException.class) intercepts first. Served as 400, never the intended 404.**

DuplicateEntryException (new — same shape)

```
class DuplicateEntryException extends RuntimeException
```

(no @ResponseStatus annotation of its own)

Also has no dedicated @ExceptionHandler — also falls **through to the same generic RuntimeException handler.** Whatever status a "duplicate entry" case should return (409 Conflict?) — it's always 400 instead.

⚠ CRITICAL — two handlers return HTTP 200 OK for error conditions

handleDataIntegrityViolationException()

```
CommonResponse<Object> response = new CommonResponse<>();
response.setStatusCode(HttpStatus.BAD_REQUEST.value());
return ResponseEntity.ok(response); // HTTP 200
```

Body says "statusCode": 400 — the actual wire-level HTTP status is 200 OK. A duplicate-entry/constraint violation is indistinguishable from success by status code.

handleNoPendingRecords()

```
return ResponseEntity.ok(
new CommonResponse<>(404, ex.getMessage(), null));
```

Same pattern, second handler: HTTP 200 wrapping a body that says "statusCode": 404. Two independent handlers in the same file share this exact bug shape — not a one-off typo.

Any client that checks the HTTP status code rather than parsing the body treats both of these failure cases as success.

Minor findings

Raw exception messages surfaced to API clients

handleRuntimeException(), handleGeneralException(), and handleIllegalArgumentException() all return ex.getMessage() directly in the response body — whatever a vendor library, SQL driver, or internal exception happens to say is exposed verbatim to the API caller.

Old handler left as a comment — pattern recurs again

An earlier IllegalArgumentException handler (15 lines, "Invalid hierarchy API request" wording) sits commented out directly above the live one — the same "old version left as a comment" habit already confirmed in Modules 2, 9, and 11.

By contrast: deliberate care taken to protect Swagger/OpenAPI endpoints

handleNoHandlerFound(), handleRuntimeException(), and handleGeneralException() all explicitly check whether the request path contains "swagger" or "api-docs" and special-case the response so generic error handling doesn't break API documentation — a small but genuinely deliberate, well-reasoned piece of defensive engineering in this file.

Sound design, by contrast | Minor / dead code | Flagged bug — wrong status served | Critical — confirmed success/failure status mismatch